BAIL security

Lista Dao
Position Migrator

FINAL REPORT

March '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	XXX - Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/tree/528802d300733794377708e1b4d7e49177d1c832 https://github.com/lista-dao/lista-dao-contracts/tree/15fe97be3548c2b8b232477e23bf976d3a63a81f
Resolution 1	Moolah: https://github.com/lista-dao/moolah/commit/86b2a6a6915aa5ce3a3c54ec97a7df41f4232d9a CDP (lista-dao-contracts): https://github.com/lista-dao/lista-dao-contracts/commit/f2bc6af16a39d1bbed50b427c966f3961efced1d

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	2	1		1		
Medium	4	1		3		
Low	1			1		
Informational	10	2		8		
Governance	1			1		
Total	18	4		14		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

PositionMigrator

The PositionMigrator contract is a new contract that allows whitelisted users to atomically migrate their existing CDP positions to Moolah lending market positions. The migration is executed in a single transaction using a Moolah flashloan to repay the CDP debt, withdraw the CDP collateral, supply that collateral to Moolah, and borrow lisUSD from Moolah to repay the flashloan. Associated changes to the CDP codebase have been made to enable the PositionMigrator to operate on users' positions.

There are three different cases in the CDP collateral withdrawal step, determined by which provider is associated with the user's CDP position: SlisBNBProvider, HelioProviderV2, or no provider. Each case uses a different withdrawal path to retrieve the collateral from the CDP. CDP positions that use a different provider will not be supported by this tool.

On the Moolah side, collateral is supplied either directly to Moolah (if the target market has no provider) or through the Moolah SlisBNBProvider (if one is configured for the market). Moolah markets that use a different provider or a loan token other than lisUSD will not be supported.

Privileged Functions

- updateWhitelist
- setSupportedCollateral

Core Invariants:

INV 1: After a successful migration, the user's CDP position is empty.
INV 2: The user's position after migration is healthy, implicitly enforced by the Moolah code.
INV 3: Only whitelisted users can initiate a migration.
INV 4: Only the Moolah contract can invoke the flashloan callback.
INV 5: The migration is atomic; any failure at any step reverts the entire transaction.

Issue_01	PositionMigrator passes free collateral to releaseFor causing underflows in Provider accounting
Severity	**Medium**
Description	In PositionMigrator.sol, the total collateral to be migrated is calculated as the sum of the user's free and locked balances. This combined amount is then passed to the target CDP provider, such as ISlisBnbProviderCdp(slisBnbProviderCDP).releaseFor(). However, the SlisBNBProvider strictly separates the accounting of locked and free collateral. The internal userLp tracking system only accounts for locked collateral (as seen in _syncLp, which solely queries dao.locked()). Free collateral does not have associated LP tokens and is intended to be withdrawn separately via withdrawLeftover(), which bypasses LP burning. By passing free + locked into releaseFor(), the Migrator forces the Provider to route the free portion through _withdrawLp and _burnLp. Because the user does not possess LP tokens for the free portion of their collateral, _burnLp will attempt to burn more tokens than the user owns. This triggers an underflow (userLp[_account] -= userPart), reverting the transaction and blocking the user from migrating.
Recommendations	The PositionMigrator should only pass the locked portion to releaseFor() and handle the free portion separately, for example by implementing a new function that allows the migrator to withdraw the free portion.
Comments / Resolution	Fixed following recommendations. Note that _withdraw() in Interaction pulls from free collateral (gem) first, so if a user has both locked and free collateral, the migration will consume the free portion before unlocking from ink. Any remaining ink is left behind with zero debt. This is an equivalent state to having deposited collateral without borrowing yet, and the user can withdraw that collateral at a later stage, similar to withdrawing fully-freed gem collateral.

Issue_02	Existing Moolah position is combined with migrated position
Severity	**Medium**
Description	If a user already has a position in the target Moolah market before migrating, the migration adds further collateral and debt to the existing position. The borrow function checks _isHealthy on the combined position at the end, so if the combined position is unhealthy (e.g., the existing position was already near liquidation), the transaction reverts. However, since Moolah markets let you borrow right up to the liquidation LTV, it results in a position that can be liquidated immediately after. Furthermore, existing safe positions can have their health go down, and thus this risks more capital than the migration amount. Say the CDP had $1000 worth of collateral tokens and 879 debt, with an allowed LTV of 90%. This position is healthy, with some margin of safety. Say this position is then migrated to a Morpho market with an LTV of 80%, with a pre-existing collateral worth 100. The new position will have $1100 collateral and 879 debt. This will be allowed since the current LTV is now 79.99%, but any interest accrual will make this position liquidatable. So the user risks losing not just their migrated funds but also their pre-existing collateral.
Recommendations	Consider documenting this behavior for users, and have the UI clearly show the health of the resulting position, including any pre-existing collateral/debt in that market.
Comments / Resolution	Acknowledged and documented.

Issue_03	Incorrect deposit amount when migrating native BNB positions in case of no provider set up in Moolah
Severity	**Medium**
Description	When users migrate their native BNB CDP position, their collateral is converted into slisBNB instead. This slisBNB is then added to the Moolah market as collateral. For this flow, the user has to pass in isBnb=true. Since BNB doesn't get converted 1:1 to slisBNB, the code does a pre-post check to count the amount of actual tokens received. It uses this amount for isBnB=true flows, while in other flows it uses the raw collateralAmount value. *releasedSlisBnb = IERC20(SLISBNB).balanceOf(address(this)) - releasedSlisBnb;* *if (data.isBnb) {* *require(releasedSlisBnb >= data.minSlisBnb, "slippage too high");* *}* The issue is that when it's time to deposit into Moolah, it checks if the market has a provider. If not, it uses the raw collateralAmount value, which is incorrect for native BNB migrations. if (provider == address(0)) { MOOLAH.supplyCollateral(params, data.collateralAmount, data.onBehalf, ""); So native BNB positions can only be migrated to slisBNB markets with providers. If a user wishes to migrate to a market with no provider set up, the transaction reverts, since it tries to deposit data.collateralAmount tokens instead of the lower releasedSlisBnb amount. This can also be used maliciously to steal any stranded slisBNB tokens present in the migrator contract, since it deposits more than it actually gains.

Recommendations	Consider using data.collateralAmount for both Moolah deposit flows, and if isBnb=true, overwrite data.collateralAmount with releasedSlisBnb.
Comments / Resolution	Acknowledged, all slisBnb markets must have the slisBnb provider.

Issue_04	No choice for other providers on Moolah
Severity	**Informational**
Description	If a provider is found in the target market on Moolah, there is a check restricting it to be the slisBNBProvider. *require(provider == slisBnbProviderLending, "invalid moolah provider");* Lista CDP positions have ETH, USDT, BTCB, and various other tokens as collateral. If a provider is set up for such a token on a Moolah market, migrating into that market will not be possible due to the check above. The only provider allowed in the migrator contract is the slisBNB provider, which can prevent certain migration paths if other providers are also deployed.
Recommendations	Consider acknowledging and documenting this behaviour, since other providers are not currently deployed.
Comments / Resolution	Acknowledged.

Issue_05	Free collateral (gem) is swept into the Moolah position
Severity	**Informational**
Description	The migrator calculates the total collateral to migrate as free(token, user) + locked(token, user), which includes both the ink (collateral backing debt, subject to liquidation) and the gem (free collateral sitting in the Vat, not backing any debt). The migration supplies this entire amount to Moolah as collateral for the new borrowed position. This means that collateral which was previously free and unlocked in the CDP (not at risk of liquidation, withdrawable at any time) becomes locked into the Moolah position, where it backs debt and is subject to Moolah's liquidation behavior. Users with significant gem balances should be aware that migrating will lock this collateral into a position.
Recommendations	Consider documenting this behavior for users.
Comments / Resolution	Fixed by only migrating locked collateral.

Issue_06	releasedSlisBnb logic runs in all scenarios regardless of collateral type
Severity	**Informational**
Description	The migrator computes releasedSlisBnb by taking the difference of slisBNB.balanceOf(address(this)) before and after the CDP collateral withdrawal. This logic runs in all migration paths, including those where slisBNB has nothing to do with the CDP position. In those cases, the balanceOf calls are wasted gas.
Recommendations	Consider only computing releasedSlisBnb when isBnb == true, or restructuring the code so that balance measurements are scoped only to the paths that use them.
Comments / Resolution	Acknowledged.

Issue_07	Migration requires sufficient lisUSD liquidity for both flashloan and borrow
Severity	**Informational**
Description	The migration flashloans the full CDP debt amount in lisUSD from Moolah, then later borrows lisUSD from Moolah to repay the flash loan. Both operations draw from the same lisUSD liquidity pool. If there is not enough lisUSD in the Moolah market to service both the flashloan and the subsequent borrow, the migration will revert.
Recommendations	Keep this behavior in mind and acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_08	Migration during liquidation can leave behind collateral
Severity	**Informational**
Description	If a user's CDP position is liquidated and the user then migrates while the auction is still ongoing, the migration will operate on the reduced position. When the auction later concludes, any leftover collateral is returned to the user's gem balance in the Vat via vat.flux. This collateral is not migrated, but the user can withdraw it from the CDP system later. This is not a vulnerability since the user's funds are not lost, but users should be aware that migrating during an active liquidation auction may leave collateral behind in the CDP system.
Recommendations	Consider documenting this edge case for users.
Comments / Resolution	Acknowledged and documented.

Issue_09	CDP and Moolah use different oracles and LTV parameters
Severity	**Informational**
Description	The CDP and Moolah systems use different Oracle implementations and different LTV parameters. Users should understand the parameters of the system they are migrating to, as the two systems will behave differently.
Recommendations	Consider documenting the Oracle and LTV differences for users before they initiate a migration.
Comments / Resolution	Acknowledged.

Issue_10	Insufficient test coverage
Severity	**Informational**
Description	The migrator supports multiple paths: 1. Migration of native BNB CDP positions via provider 2. Migration via direct CDP interaction 3. Migration via slisBNB CDP provider The current fork tests do not cover all these scenarios. Ensure proper coverage of all these scenarios before deployment.
Recommendations	Ensure proper fork-testing coverage of all these scenarios before deployment.
Comments / Resolution	Acknowledged. QA team is currently running more tests.

Issue_11	0 loan positions cannot be migrated
Severity	**Informational**
Description	Some users can have positions with no loans in the CDP system. These users will not be able to use the migration tool, since the flashloan requires a non-zero amount.
Recommendations	During migration, include special instructions for removing funds manually for users with 0 loan CDP positions.
Comments / Resolution	Acknowledged.

Issue_12	PositionMigrator is incompatible with Moolah broker markets
Severity	**Low**
Description	The PositionMigrator finalizes a user's migration by borrowing LISUSD from the target Moolah market on the user's behalf to repay the flash loan. However, the Moolah contract enforces strict access control on broker markets: if a market has an assigned broker, both the msg.sender and the receiver must be the broker contract itself (require(msg.sender == broker && receiver == broker)). Because the migrator passes its own address as the sender and receiver, attempting to migrate a position into any broker-enabled market will consistently revert with a NOT_BROKER error.
Recommendations	Consider acknowledging this issue or implement functionality that allows migrating to a broker-enabled market.
Comments / Resolution	Acknowledged.

Issue_13	Migration blocked if target Moolah provider reaches capacity limit
Severity	**Informational**
Description	During migration, the PositionMigrator withdraws collateral from the CDP and supplies it to Moolah via ISlisBnbProvider(provider).supplyCollateral(). If the MPC wallet capacity limit is reached for all mpc wallets, attempting to supply collateral beyond this limit will revert. Consequently, users are entirely blocked from migrating their CDP positions.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Interaction

The Interaction contract is the primary entry point for users managing CDP positions. This part of the review was performed as a differential audit, and any issues present in the original implementation outside the modified code are not in scope. The changes in scope are the addition of the withdrawFor function (which allows the migrator to withdraw collateral on behalf of a user), the new migrator view function, and the removal of the initialize function since re-initialization is not needed for the proxy upgrade.

Privileged Functions

- withdrawFor

Core Invariants:

INV 1: Only the migrator can call withdrawFor

Issue_14	No-provider CDP withdrawal path always reverts
Severity	**High**
Description	The withdrawFor function in the Interaction contract is the migration path for CDP positions that have no associated provider (i.e., helioProviders[token] == address(0)). This function passes msg.sender (the migrator contract) as the caller argument into the internal _withdraw function. The _withdraw function contains a check that requires caller == participant when there is no provider set for the token. Since the caller is the migrator address and the participant is the user, this check always fails, causing all no-provider migrations to revert.
Recommendations	Move the caller == participant check out of _withdraw and into the public withdraw function, so that withdrawFor (which is already access-controlled to the migrator) can bypass it for no-provider tokens. Also, ensure that this code path is tested.
Comments / Resolution	Fixed following recommendation.

Issue_15	The migration feature is not functional in the current implementation
Severity	**High**
Description	The migrator view function in the Interaction contract is hardcoded to return address(0) with an open TODO comment. Since all migration functions check msg.sender == migrator(), the code as written will always revert. This is planned to be updated before deployment, but as it stands, the code that would be deployed is not functional.
Recommendations	Consider altering the deployment process so that the migrator address is set in storage during the upgrade itself (e.g., via a function called as part of the upgrade transaction). This would allow the complete flow to be properly tested and audited.
Comments / Resolution	Acknowledged, will update the address later.

Issue_16	Mismatched return type for withdrawFor between interface and implementation
Severity	**Informational**
Description	The IInteraction interface defines withdrawFor with an int256 return type, while the Interaction implementation returns a uint256. Because the PositionMigrator contract ignores this return value entirely, and Interaction doesn't explicitly inherit the interface, this mismatch won't cause execution or compilation errors. However, it introduces technical debt and could lead to silent casting issues if future contracts rely on the interface's return value.
Recommendations	Update IInteraction.sol so withdrawFor returns uint256 to match the implementation.
Comments / Resolution	Fixed following recommendations.

Issue_17	Migrator can be set to any address to drain the protocol
Severity	**Governance**
Description	The dao contract can have its migrator value set to any address before upgrade deployment. The migrator contract itself is also upgradable and can be changed to be malicious later. This address will then have the ability to withdraw any position at any time, and thus poses a centralization risk to the system.
Recommendations	Ensure the migrator address is correctly set to the migrator contract address at upgrade deployment and not some malicious EOA. Set up multisig wallets to control protocol upgrades to defend against private key leaks.
Comments / Resolution	Acknowledged.

HelioProviderV2

HelioProviderV2 is the CDP provider for BNB collateral. This part of the review was performed as a differential audit, and any issues present in the original implementation outside the modified code are not in scope. The change in scope is the addition of releaseInTokenFor, which allows the migrator to withdraw a user's BNB-denominated collateral in the form of slisBNB.

Core Invariants:

INV 1: Only the migrator can call releaseInTokenFor

No issues were identified in this contract.

SlisBNBProvider

SlisBNBProvider is the CDP provider for slisBNB collateral. This part of the review was performed as a differential audit, and any issues present in the original implementation outside the modified code are not in scope. The change in scope is the addition of releaseFor, which allows the migrator to withdraw a user's slisBNB collateral from the CDP by burning their LP tokens and transferring slisBNB directly to the migrator.

Core Invariants:

INV 1: Only the migrator can call releaseFor

Issue_18	_withdrawLp incorrectly syncs msg.sender instead of _account causing accounting corruption and underflows during migration
Severity	**Medium**
Description	The BaseTokenProvider contract contains a critical accounting flaw in its internal _withdrawLp function. It incorrectly hardcodes msg.sender when attempting to synchronize a user's LP balance, rather than using the passed _account parameter. Historically, this bug remained dormant because the only way to trigger _withdrawLp was through the standard release() function. In release, a user withdraws their own funds, meaning msg.sender and _account are exactly the same address. In that context, the hardcoded msg.sender synced the correct user. However, the introduction of the releaseFor function in SlisBNBProvider exposed this issue. releaseFor was added to allow the PositionMigrator contract to withdraw collateral on a user's behalf. When a user migrates their position, the PositionMigrator calls releaseFor(userAddress, amount). During this execution msg.sender is the Migrator contract and _account is the user. As a result, _withdrawLp() attempts to sync the balance of the Migrator contract (which is zero) and completely fails to sync the user's balance. If the exchangeRate or userLpRate has changed since the user last

	interacted with the protocol, their userLp[_account] balance will be stale. When _burnLp subsequently executes, it calculates the LP to burn based on the new rates, but subtracts it from the stale balance. This will either trigger an arithmetic underflow, reverting the entire migration transaction, or corrupt the user's internal accounting state.
Recommendations	Update _withdrawLp in BaseTokenProvider to synchronize the correct account parameter.
Comments / Resolution	Acknowledged.